



You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Overview

Noctalia Shell features a powerful plugin system that allows you to extend the shell with custom functionality. Plugins are written in QML and can integrate seamlessly with the shell's UI and services.

Quick Start

New to plugin development? Jump straight to the [Getting Started guide](#) to create your first plugin in minutes!

What Can Plugins Do?

Plugins can provide several types of components, each serving different purposes:

Bar Widgets

Add custom widgets to the top/bottom bar that provide quick access to information and actions.

Use cases:

- Display system information (CPU, memory, network)
- Show custom indicators (notifications, status)
- Quick action buttons (launcher, shortcuts)
- Status monitors (battery, time, weather)

Learn more: [Bar Widget Development](#)

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Add custom widgets to your desktop background. They provide at-a-glance information and can be positioned and scaled by the user in edit mode.

Use cases:

- Clock, MediaPlayer
- Weather forecast
- Audio visualizers

Learn more: [Desktop Widget Development](#)

Control Center Widgets

Add quick action buttons to the Control Center panel. These are typically simple buttons that toggle panels or perform actions.

Use cases:

- Quick toggle actions
- Open plugin panels
- System shortcuts

Learn more: [Control Center Widget Development](#)

Launcher Providers

Extend the app launcher with custom search sources, command handlers, and browsable content.

Use cases:

- Custom search sources (emoji, kaomoji, snippets)
- Command handlers (>kaomoji, >todo)

- Category-based browsing
- Clipboard history, bookmarks

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Panels

Create full-screen overlay panels that can be opened from bar widgets or triggered programmatically.

Use cases:

- Detailed information displays
- Complex UI interactions
- Configuration interfaces
- Content browsers and lists

Learn more: [Panel Development](#)

Main Component

Run background logic and handle IPC (Inter-Process Communication) commands. Optional component for plugins that need background processing.

Use cases:

- Process external commands via IPC
- Monitor system events
- Perform background tasks
- Integrate with external services

Learn more: [IPC Guide](#)

Settings UI

Provide a settings interface that integrates with Noctalia's settings panel, allowing users to configure your plugin.

Use cases:

Plugin configuration

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Plugin customization

- Feature toggles
- Preferences management

Learn more: [Settings UI Guide](#)

Plugin Architecture

Plugins in Noctalia follow a modular architecture:

```
your-plugin/
├─ manifest.json      # Plugin metadata (required)
├─ preview.png        # Preview image for plugin gallery (optional, recommended)
├─ Main.qml           # Background logic with IPC handlers (optional)
├─ BarWidget.qml      # Bar widget component (optional)
├─ DesktopWidget.qml  # Desktop widget component (optional)
├─ ControlCenterWidget.qml # Control center button (optional)
├─ LauncherProvider.qml # Launcher search provider (optional)
├─ Panel.qml          # Panel overlay component (optional)
├─ Settings.qml       # Settings UI component (optional)
├─ i18n/              # Translations (optional)
│   └─ en.json
│   └─ es.json
├─ README.md          # Plugin documentation
└─ settings.json       # User saved settings (should not be committed to the repository)
```

Preview Images

Add a `preview.png`, `preview.jpg`, `preview.jpeg`, or `preview.webp` file to your plugin folder to display a preview image in the plugin gallery. The preview image should showcase what your plugin does or how it looks. Recommended size: 800x600px or similar aspect ratio. If no preview image is provided, a placeholder will be shown instead.

Plugin Lifecycle

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

-
2. **Registration:** PluginRegistry scans the folder and validates the manifest
 3. **Enabling:** User enables the plugin in settings
 4. **Loading:** PluginService loads the plugin components and injects the Plugin API
 5. **Running:** Plugin components receive pluginApi property with access to services
 6. **Settings:** Users can configure the plugin through the integrated settings UI
 7. **Unloading:** When disabled, the plugin is cleanly unloaded
 8. **Uninstallation:** Plugin folder is removed from disk

Plugin API

Every plugin component receives a pluginApi object that provides access to plugin functionality and Noctalia services.

Core Properties

- **pluginId** - Unique plugin identifier
- **pluginDir** - Plugin directory path
- **pluginSettings** - User settings object (read/write)
- **manifest** - Plugin manifest data
- **currentLanguage** - Current UI language code
- **mainInstance** - Reference to the instantiated Main.qml component (null if not loaded)
- **barWidget** - Reference to the bar widget Component (null if not provided)

Core Functions

- **saveSettings()** - Persist settings to disk
- **openPanel(screen, buttonItem?)** - Open the plugin's panel (optionally near buttonItem)
- **closePanel(screen)** - Close the plugin's panel

- **togglePanel(screen, buttonItem?)** - Toggle the plugin's panel open/closed (optionally near buttonItem)

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

- **trp(key, count, ...)** - Translate with plurals
- **hasTranslation(key)** - Check if translation exists

Noctalia Services

Plugins have full access to Noctalia's service ecosystem:

qs.Commons - Core utilities

- Settings - Global Noctalia settings
- I18n - Internationalization
- Logger - Logging utilities
- Style - Design system constants
- Color - Theme-aware colors

qs.Services.UI - UI services

- ToastService - Toast notifications
- PanelService - Panel management
- TooltipService - Tooltip display

qs.Services.System - System services

- AudioService - Audio control
- BatteryService - Battery information
- NetworkService - Network status
- And many more...

qs.Widgets - UI components

- NButton, NIcon, NText, NTextInput
- NScrollView, NColorPicker, and more

Full API Reference

For complete API documentation, see the [Plugin API Reference](#).

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Learning Path

Follow this recommended path to master plugin development:

1. [Getting Started](#) - Create your first plugin
 - Set up your development environment
 - Create a basic bar widget
 - Understand the plugin structure
2. [Manifest Reference](#) - Understand plugin configuration
 - Learn all manifest fields
 - Configure entry points
 - Set up default settings
3. [Bar Widget Development](#) - Build bar widgets
 - Create interactive widgets
 - Handle user interactions
 - Style with Noctalia's design system
4. [Desktop Widget Development](#) - Create desktop widgets
 - Draggable, scalable widgets
 - At-a-glance information displays
5. [Control Center Widget](#) - Add control center buttons
 - Quick action buttons
 - Panel toggle buttons
6. [Launcher Provider](#) - Extend the launcher
 - Custom search sources
 - Command handlers

- Category-based browsing

7. [Panel Development](#) - Create overlay panels

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

- Handle panel lifecycle
- Create complex UIs

8. [Plugin API Reference](#) - Master the API

- Complete API documentation
- Service integration
- Advanced patterns

9. [IPC Guide](#) - Background processing

- Create IPC handlers
- Handle external commands
- Integrate with scripts and keybindings

10. [Additional Topics](#)

- [Settings UI](#) - Configuration interfaces
- [Translations](#) - Internationalization

Example Plugins

Explore these example plugins in the [official repository](#) to see real-world implementations:

- **hello-world** - Demonstrates all plugin features (bar widget, desktop widget, control center widget, panel, settings, IPC)
- **kaomoji-provider** - Launcher provider with command mode, categories, and database loading
- **privacy-indicator** - Shows when mic/camera/screen sharing is active

Next Steps

Ready to start building? Head to the [Getting Started guide](#) to create your first plugin in just a few minutes!

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)
